Performance tips

SWAT+ is a serial Fortran program. There is no MPI, no OpenMP. The run time is set by the simulation length, the number of objects (HRUs, channels, aquifers), the time step, and how much output is written. The build is parallel; the model is not.

Build with `-j`

CMake passes the parallel-jobs flag to the underlying build tool. Use it on the build step:

```
cmake -B build
cmake --build build -j 8
```

`-j 8` runs eight compile jobs at once. Use a value close to your physical core count. The build is CPU-bound; oversubscribing the I/O does not help.

A clean build of SWAT+ on a recent laptop takes a couple of minutes with `-j 8` and around ten minutes with `-j 1`.

Choose an optimized build type

For production runs, build in Release mode (debug symbols and runtime checks off):

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

This is the default in the SWAT+ `CMakeLists.txt`. If you have configured a Debug build, expect runs to be five to ten times slower.

For development, use Debug or RelWithDebInfo so floating-point checks are enabled.

Daily versus sub-daily routing

`time.sim` field `step`:

- `step = 0`: daily. Fastest.
- `step > 0`: sub-daily. Required for Green-Ampt infiltration (`gampt = 1` in `codes.bsn`) and for sub-daily channel routing.

Sub-daily routing multiplies the inner-loop work by the number of sub-daily steps per day. A 46-year daily run that finishes in ten seconds will take several minutes at hourly. Use sub-daily only when the science needs it. Green-Ampt and hourly stage data are the most common reasons.

References: time.sim, codes.bsn.

The cost of enabling many outputs

Every row in `print.prt` that is set to `daily` writes one record per object per day. The cost grows with the simulation length, the number of objects, and the number of rows.

Concretely:

- 46 years of daily output for 12 HRUs and 10 output rows is 12 * 46 * 365 * 10 = around 2 million records. The model produces these in seconds, but parsing them downstream can take longer than the simulation itself.
- The `aa` (average-annual) interval is essentially free. One record per object at end of run.

- `mon` (monthly) is 12 per year. `yr` (yearly) is 1 per year.

If you only need average-annual results, leave everything at `aa` (as the Ames_sub1 default does). Turn `daily` on only for the variables you will plot.

The CSV-output flag (`csvout = y` in `print.prt`) also adds cost: every output is written twice (once as fixed-width, once as CSV). Disable when not needed.

Reference: print.prt.

Object count

Run time scales roughly linearly with the number of HRUs, channels, aquifers, and routing units in the project. A 12-HRU project (Ames_sub1) finishes in a few seconds. A 5000-HRU regional project takes minutes per simulated decade.

If you are iterating on inputs, prototype on a small subset and scale up only when the inputs are stable.

I/O

SWAT+ reads inputs once at the start and writes outputs progressively during the run. Putting the project folder on a local SSD instead of a network share matters more than CPU when many outputs are enabled.

Profiling

Compile with `-pg` (gfortran) or the Intel compiler's profile flags, run a representative scenario, and inspect with `gprof` or the Intel tools. Most time in a typical project goes to `pl_grow` (plant growth), the soil-water routines, and the channel routing. Adding more daily output rows shifts the balance toward the I/O routines.

Summary

Lever	Effect
`cmake --build build -j N`	Build time, no run-time effect.
`CMAKE_BUILD_TYPE=Release`	Five to ten times faster runs.
`step = 0` in `time.sim`	Daily, fastest.
`step > 0` in `time.sim`	Sub-daily, multiplies inner-loop cost.
Many `daily` rows in `print.prt`	More disk and downstream parsing cost.
Use `aa` only	Cheapest output mode.
Local SSD vs network share	Real difference when many outputs are on.